

Decentralized Computing and Blockchains 2025/26

Project 3 – DEX-NFT Pawning Dapp

Version 1.1

This project aims to implement a Dapp (web interface, smart contracts, and server state) to manage ERC20 [1] (fungible) and ERC721 [2] (non-fungible) tokens. The Dapp must implement an NFT marketplace and allow users to manage DEX (a fungible token) and lend ether using DEX and NFTs as collateral.

Project Description

This time, there's no detailed specification for the project. Below there's a list of required functionalities which the groups are free to implement them in the way they wish, as long as they can defend their choices in the end.

- 1) DEX marketplace: the Dapp must allow users to **buy** and **sell** DEX tokens from the Dapp smart contract using ether.
- 2) Ether lending using DEX as collateral: just like in project 2, a user can lend up to half of the value of a DEX collateral in ether. The user pays an interest value each payment cycle until returning the loan and the loan can be finalized early by paying a fixed tax.
- 3) NFT marketplace: the Dapp must allow users to **create** (mint) and **destroy** (burn) NFTs. Moreover, it allows users to **buy** and **sell** NFTs from other users (the Dapp doesn't own NFTs) using ether and DEX tokens (as required by the seller), and the Dapp can make the conversion if needed. **The Dapp owner gets 5% of the value of each NFT sold.**
- 4) NFT auctions: users can mark their NFTs for selling for a fixed price or in an auction, with a minimum price and a maximum wait time.
- 5) Ether lending using NFT requests as collateral: users can ask for a loan by providing an NFT (instead of a DEX) as collateral. Since the Dapp doesn't own NFTs, this loan is only possible if another user provides the DEX to back it, and the Dapp gets the NFT back if the user doesn't pay. In this type of loan, the user providing the DEX gets half of the interest or fixed tax paid by the lender.
- 6) Administrator console: The smart contract owner must have an administrator console to define the Dapp's parameters.

Additional Remarks

Some additional remarks:

- All the system's security must be anchored in the smart contracts.

- The smart contracts must be as secure as possible. At a minimum, students must use AI agents to improve the security of their contracts (e.g., ask their developer agent: *"Is this contract secure?"*).
- Although the project can be implemented in a single large smart contract, it is recommended to use at least two contracts (one for the DEX and another for NFTs).
- Although it is possible to implement the work with only the web front end and the smart contracts, it is advised to include server-side state to manage additional information about users and NFTs.
- Unless clearly justified, the project should be completed using the techniques and software discussed in previous classes (smart contract programming libraries, Hardhat, etc.).
- Although AI is expected to be used in the project, the groups are fully responsible for all delivered code and must be able to explain everything they present.

Project Delivering

The delivery of the project should be a zip file containing:

- The identification of the group and the students
- All the code for the project
- Instructions for running the code
- A report with up to 1000 words plus figures explaining the architecture and additional details about the functionalities of the Dapp.
- The report must also include an additional section detailing how AI was used in the project. This section does not count towards the 1000-word limit.

Deadline

The project must be delivered on the course Moodle on May 28th (11:00hs).

References

- [1] <https://ethereum.org/en/developers/docs/standards/tokens/erc-20/>
 [2] <https://ethereum.org/en/developers/docs/standards/tokens/erc-721/>